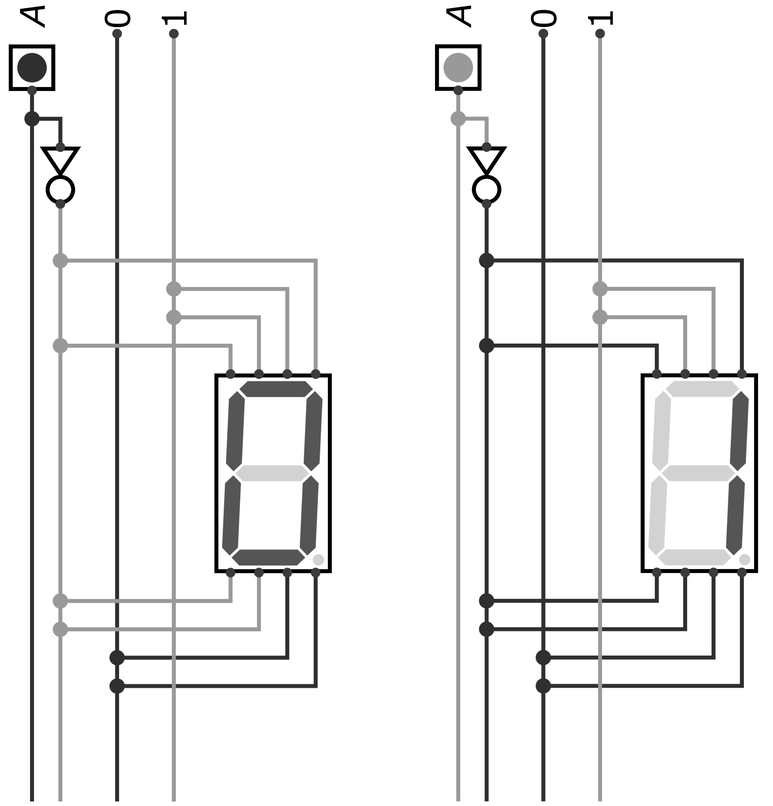


Combinational Circuits

Alex Breuer

Overview and Schedule

- A collection of combinational circuits is used commonly in hardware designs, e.g., multiplexers or shifters
- Arithmetic circuits allow us to perform arithmetic operations in hardware, e.g., additions or multiplications
- Goals:
  - Get familiar with combinational logic by studying a few of common combinational building blocks
  - Learn about arithmetic circuits and some optimizations



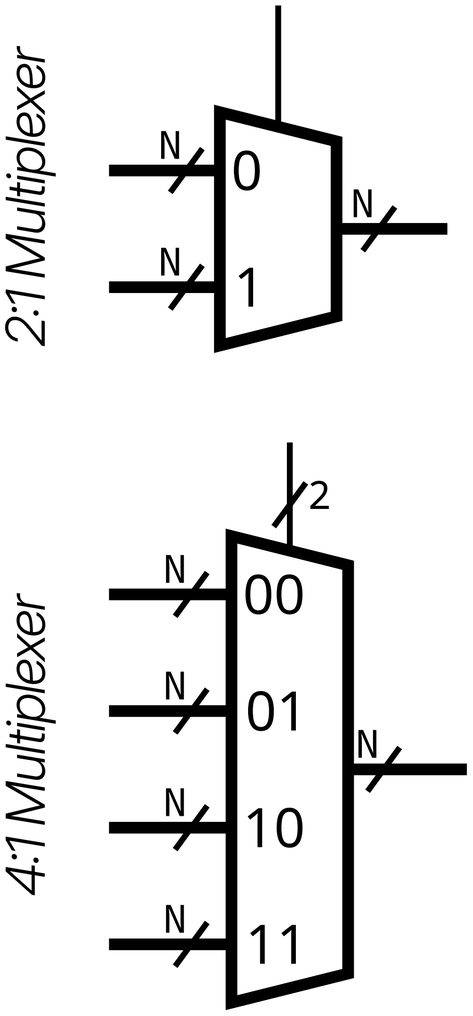
Simulation of a 1:8 decoder targeting a seven-segment display.

Multiplexers

- Selects single output from one of the inputs based on control signal(s)
- Examples:
  - 2:1 multiplexer:
    - Two N-bit inputs
    - One-bit control signal
    - One N-bit output
  - 4:1 multiplexer:
    - Four inputs
    - Two-bit control
    - One N-bit output

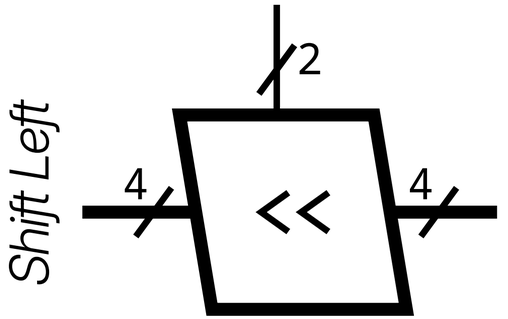
A	B	S	C
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

Truth table of a 2:1 multiplexer with 1-bit inputs.

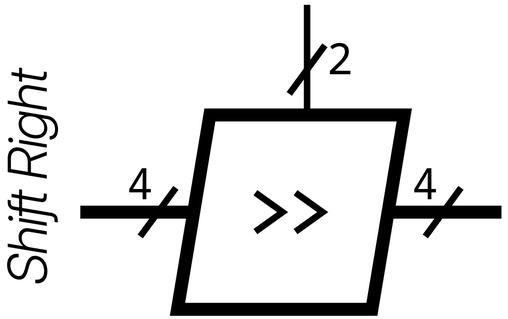


Logical Shifters

- Operator “<<” logically shifts bits to the left:
  - Bits shifted off the left side are discarded
  - Vacant bits on the right side are zero-filled
  - Example (5-bit unsigned): 10110 << 2 = 11000
- Operator “>>” logically shifts bits to the right:
  - Bits shifted off the right side are discarded
  - Vacant bits on the left side are zero-filled
  - Example (5-bit unsigned): 10110 >> 2 = 00101
- For unsigned N-bit values, shifting left by k multiplies by 2^k modulo 2^N; shifting right divides by 2^k, rounding down
- Idea for design: use multiplexers with shift amount as control



Logical shift left with a 4-bit input and output plus 2-bit shift amount.



Logical shift right with a 4-bit input and output plus 2-bit shift amount.

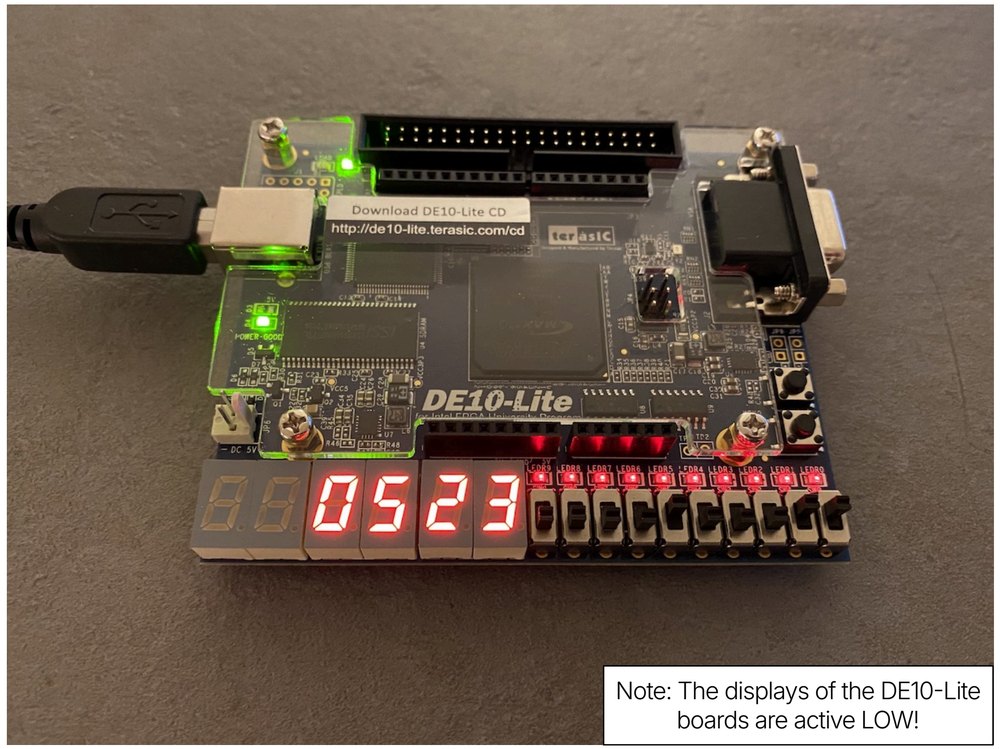
Decoders

- Converts N input bits to 2^N distinct output elements
- 2:4 decoder example:

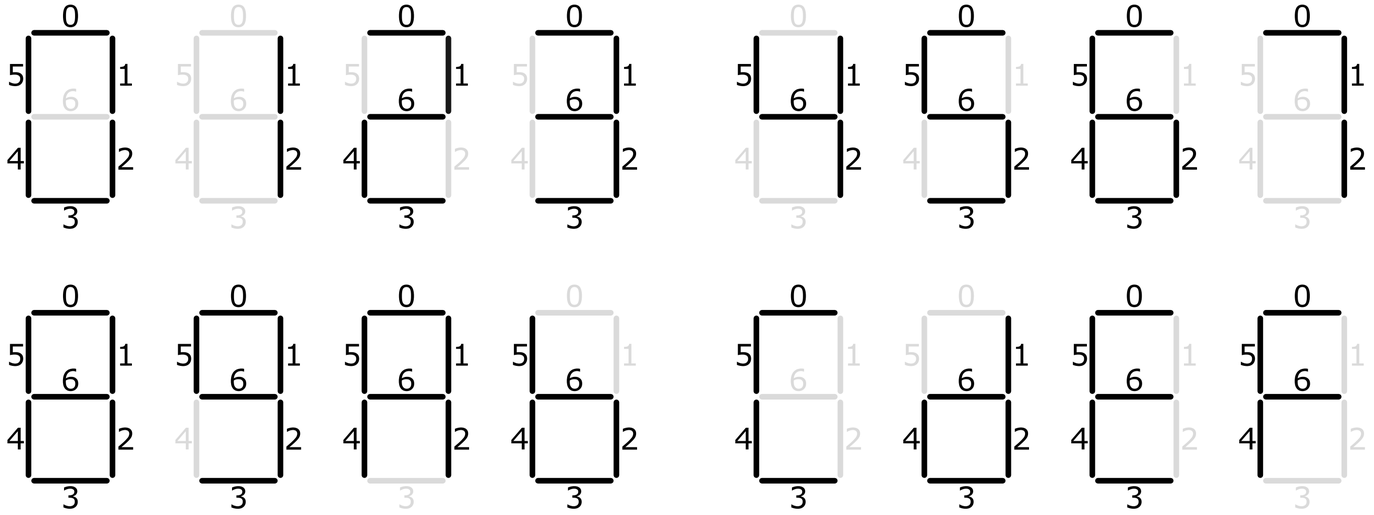
A	B	X0	X1	X2	X3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- We'll require a 4:7 decoder in the labs to use the FPGAs' seven-segment displays:

A	B	C	D	X0	X1	X2	X3	X4	X5	X6
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0							
...	...	...	...	...	...	...	...	...	...	...

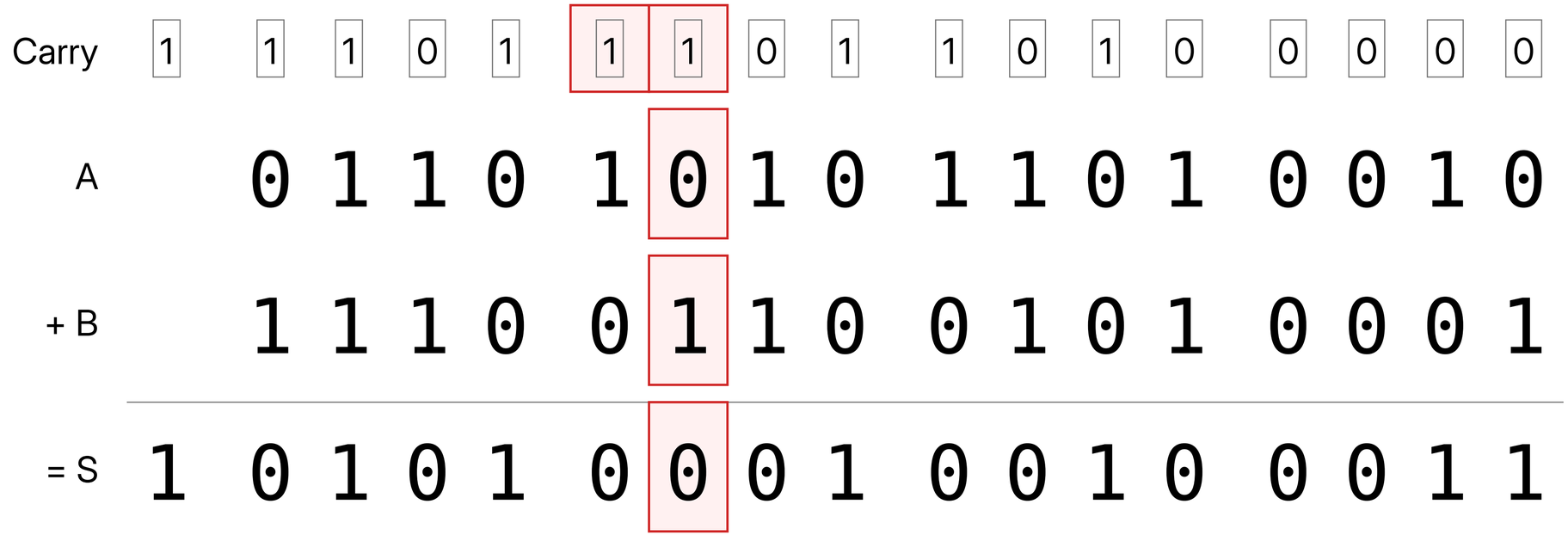


Note: The displays of the DE10-Lite boards are active LOW!



Arithmetic Circuits: Full Adder

Binary Addition



Full adder at bit 10: $C_{in} + A + B = S + 2C_{out}$

Full Adder

Truth table of the full adder.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
0	0	1	1	0
1	1	0	0	1
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

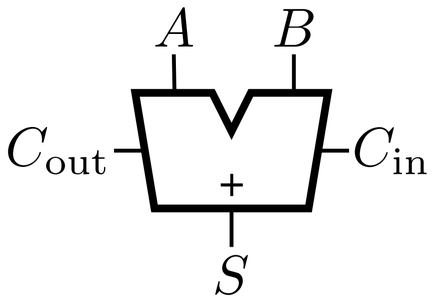
$$S = A \oplus B \oplus C_{in},$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Arithmetic Circuits: Carry-Propagate Adders

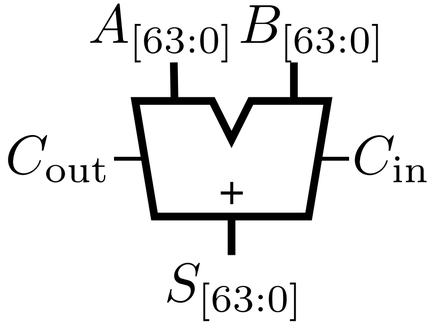
Note: we are purposely breaking with our rule that inputs are left/top and outputs right/bottom.

Ripple-Carry Adder

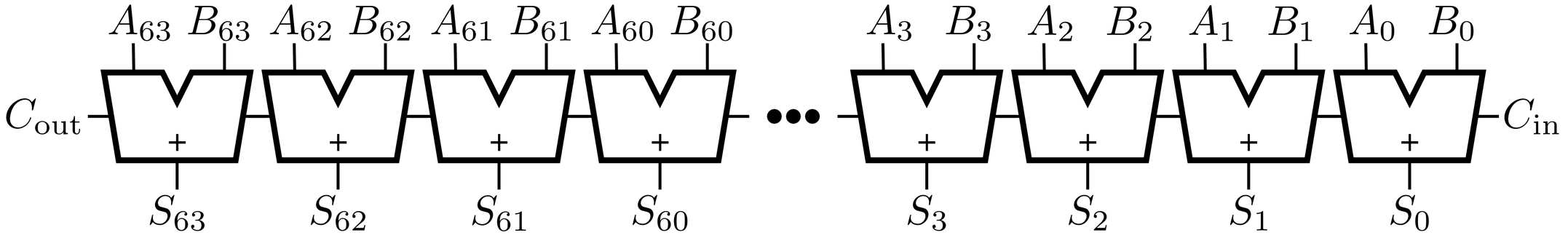
- Ripple-carry adder: Simply chain a series of full adders to obtain an adder which operates on more than one bit
- Advantage: Simple
- Disadvantage: Delay bound by carry which has to "ripple" through the entire ripple-carry adder, i.e. $t_{RCA} = N \cdot t_{FA}$ where N is the number of bits



Full Adder



Carry-Propagate Adder



64-bit Ripple-Carry Adder

Carry Generation and Propagation

- Idea: Delay of ripple-carry adder bound by carry propagation. Add extra logic to shorten this critical path!
- Introduce additional signals:
  - Generate (G) signal: Adder generates carry out signal independent of carry in
  - Propagate (P) signal: Adder produces carry out signal if there's a carry in
- As equations:
  - $G = A \cdot B$
  - $P = A + B$

Full Adder

$$S = A \oplus B \oplus C_{in}$$

$$\begin{aligned} C_{out} &= AB + AC_{in} + BC_{in} \\ &= G + PC_{in} \end{aligned}$$

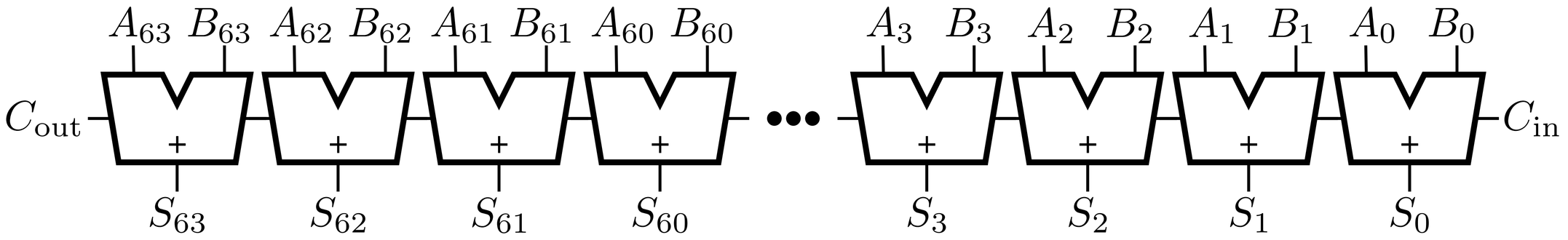
Ripple-Carry Adder: Carry Generation and Propagation

$$G_i = A_i B_i$$
$$P_i = A_i + B_i$$

$$S_0 = A_0 \oplus B_0 \oplus C_{in}$$
$$C_0 = A_0 B_0 + A_0 C_{in} + B_0 C_{in}$$
$$= G_0 + P_0 C_{in}$$

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$
$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$
$$= G_i + P_i C_{i-1}$$

$$\forall 1 \leq i < N$$



64-bit Ripple-Carry Adder

Carry-Lookahead Logic

$$C_i = G_i + P_i C_{i-1}$$

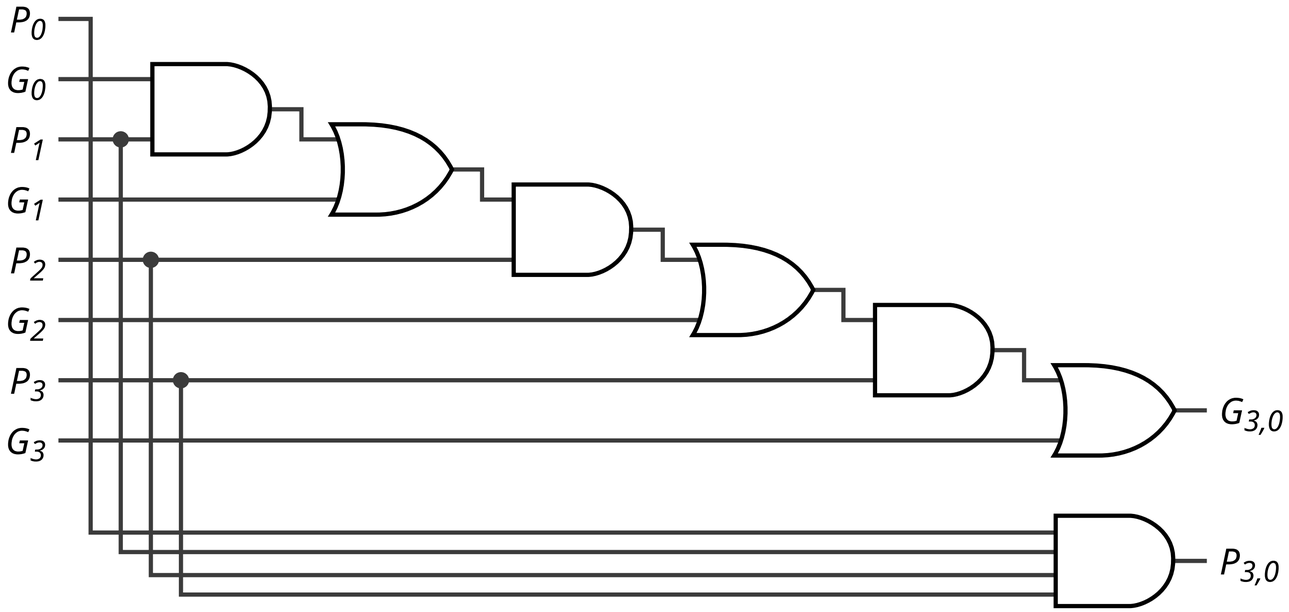
$$C_0 = \underbrace{G_0}_{G_{0,0}} + \underbrace{P_0}_{P_{0,0}} C_{in}$$

$$\begin{aligned} C_1 &= G_1 + P_1 C_0 \\ &= G_1 + P_1(G_0 + P_0 C_{in}) \\ &= \underbrace{G_1 + P_1 G_0}_{G_{1,0}} + \underbrace{P_1 P_0}_{P_{1,0}} C_{in} \end{aligned}$$

$$\begin{aligned} C_2 &= G_2 + P_2 C_1 \\ &= G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_{in}) \\ &= \underbrace{G_2 + P_2(G_1 + P_1 G_0)}_{G_{2,0}} + \underbrace{P_2 P_1 P_0}_{P_{2,0}} C_{in} \end{aligned}$$

$$\begin{aligned} C_3 &= G_3 + P_3 C_2 \\ &= G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0) + P_2 P_1 P_0 C_{in}) \\ &= \underbrace{G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0))}_{G_{3,0}} + \underbrace{P_3 P_2 P_1 P_0}_{P_{3,0}} C_{in} \end{aligned}$$

4-bit CLA Logic

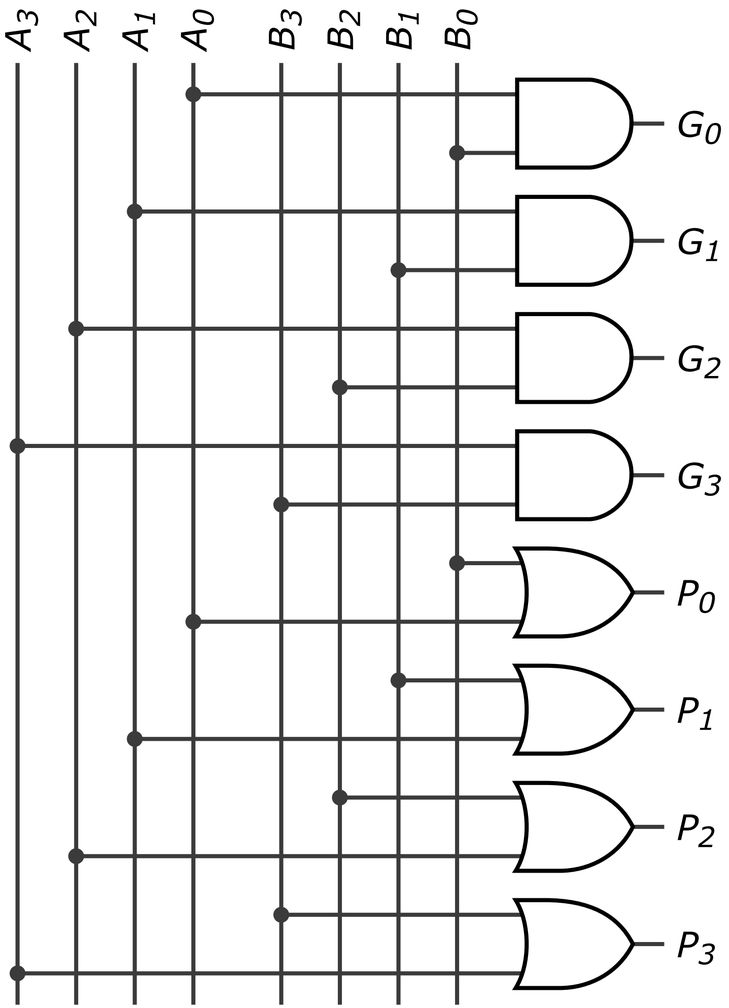


$$
\begin{aligned}
C_3 &= G_3 + P_3 C_2 \\
&= G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0) + P_2 P_1 P_0 C_{in}) \\
&= \underbrace{G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0))}_{G_{3,0}} + \underbrace{P_3 P_2 P_1 P_0}_{P_{3,0}} C_{in}
\end{aligned}
$$

$$
G_{i+k,i} = G_{i+k} + P_{i+k}\left(G_{i+k-1} + P_{i+k-1}\left(\ldots\left(G_{i+2} + P_{i+2}\left(G_{i+1} + P_{i+1} G_i\right)\right)\ldots\right)\right)
$$

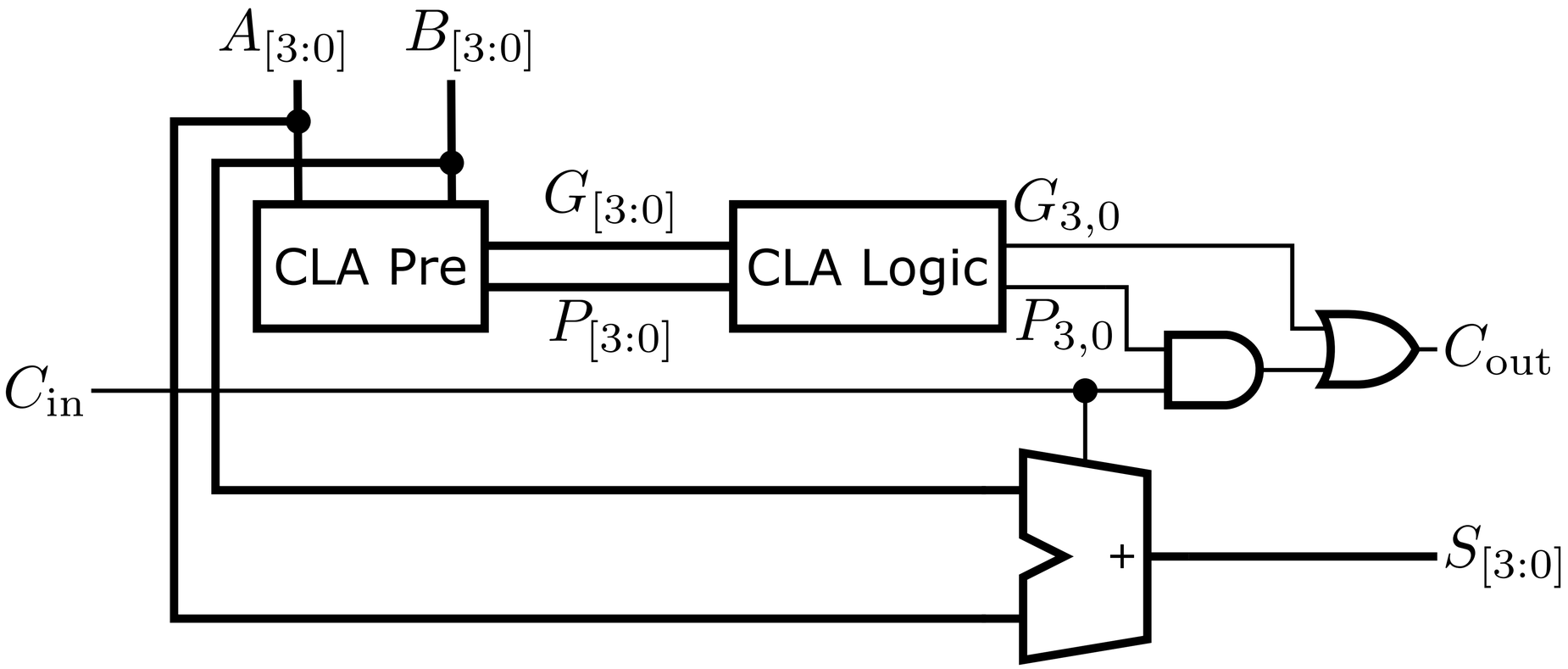
$$
P_{i+k,i} = P_{i+k} P_{i+k-1} \ldots P_{i+1} P_i
$$

4-bit CLA Pre

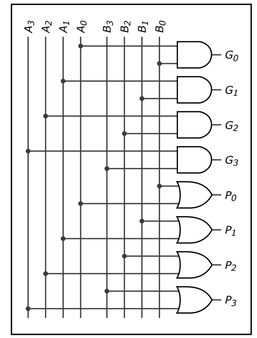


$$G_i = A_i B_i$$
$$P_i = A_i + B_i$$

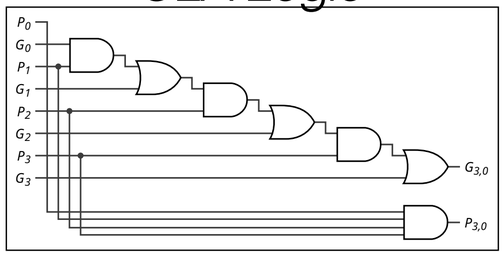
4-bit CLA Block



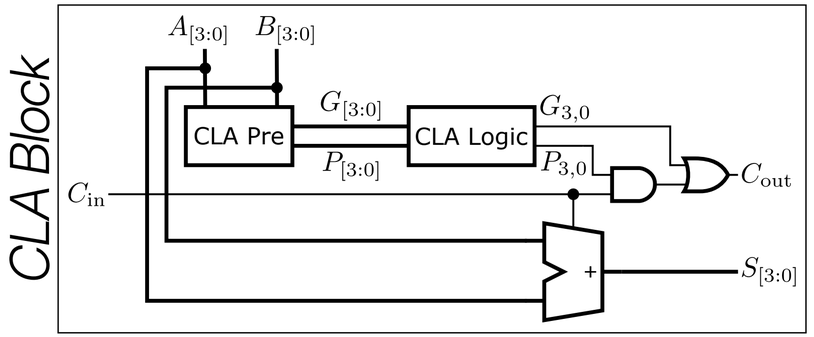
CLA Pre

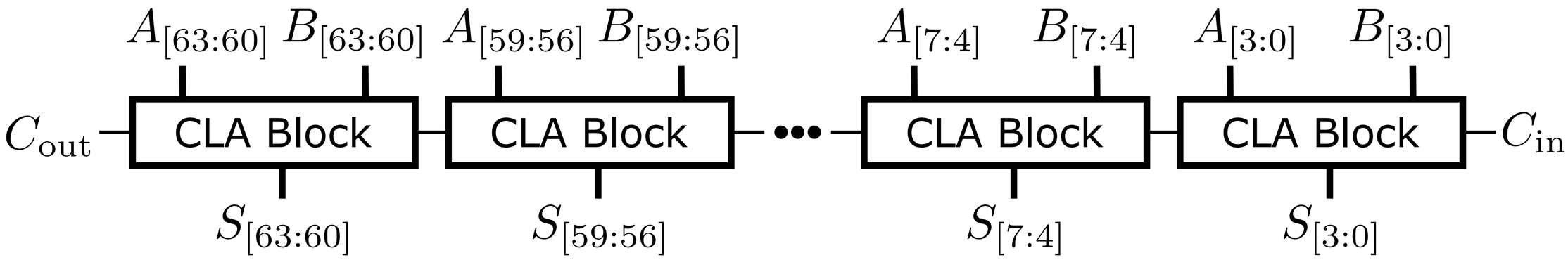


CLA Logic



64-bit Carry-Lookahead Adder (4-bit blocks)



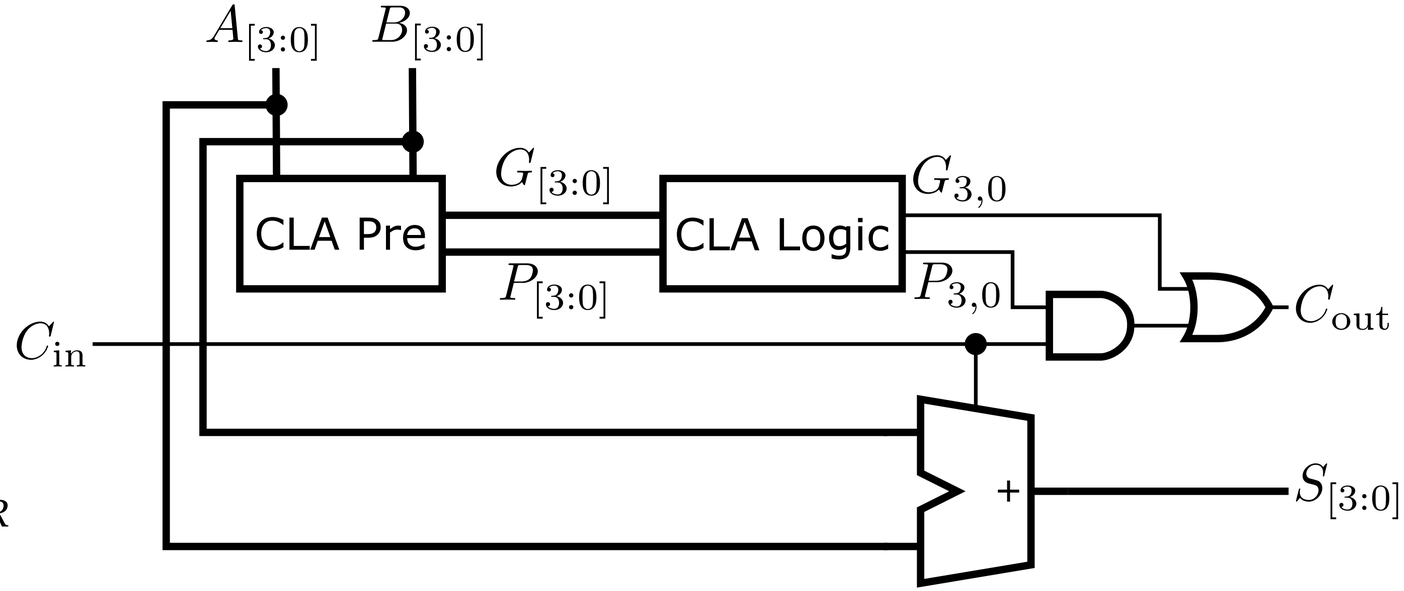


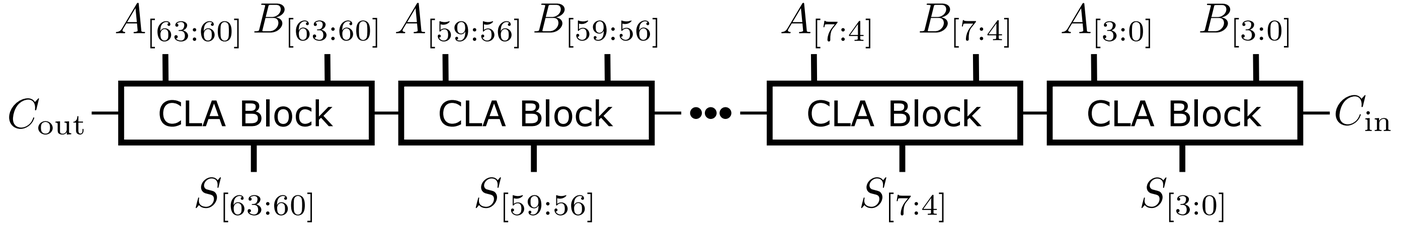
64-bit CLA Adder: Timing Analysis

- Critical path: $A_{[3:0]}$ or $B_{[3:0]}$ to $S_{[63:60]}$
- CLA adder works on all “CLA Pre” and “CLA Logic” blocks in parallel, i.e. we only have to account for delays only once (first block):

  $$t_{CLA\ Pre} + t_{CLA\ Logic} + t_{AND/OR} = 8 \cdot t_{AND/OR}$$

- For the 14 CLA blocks in the middle, the critical path goes through a single AND and a single OR gate: $14 \cdot (t_{AND} + t_{OR})$
- For the last block, we have to go through the four chained full adders: $4 \cdot t_{FA}$





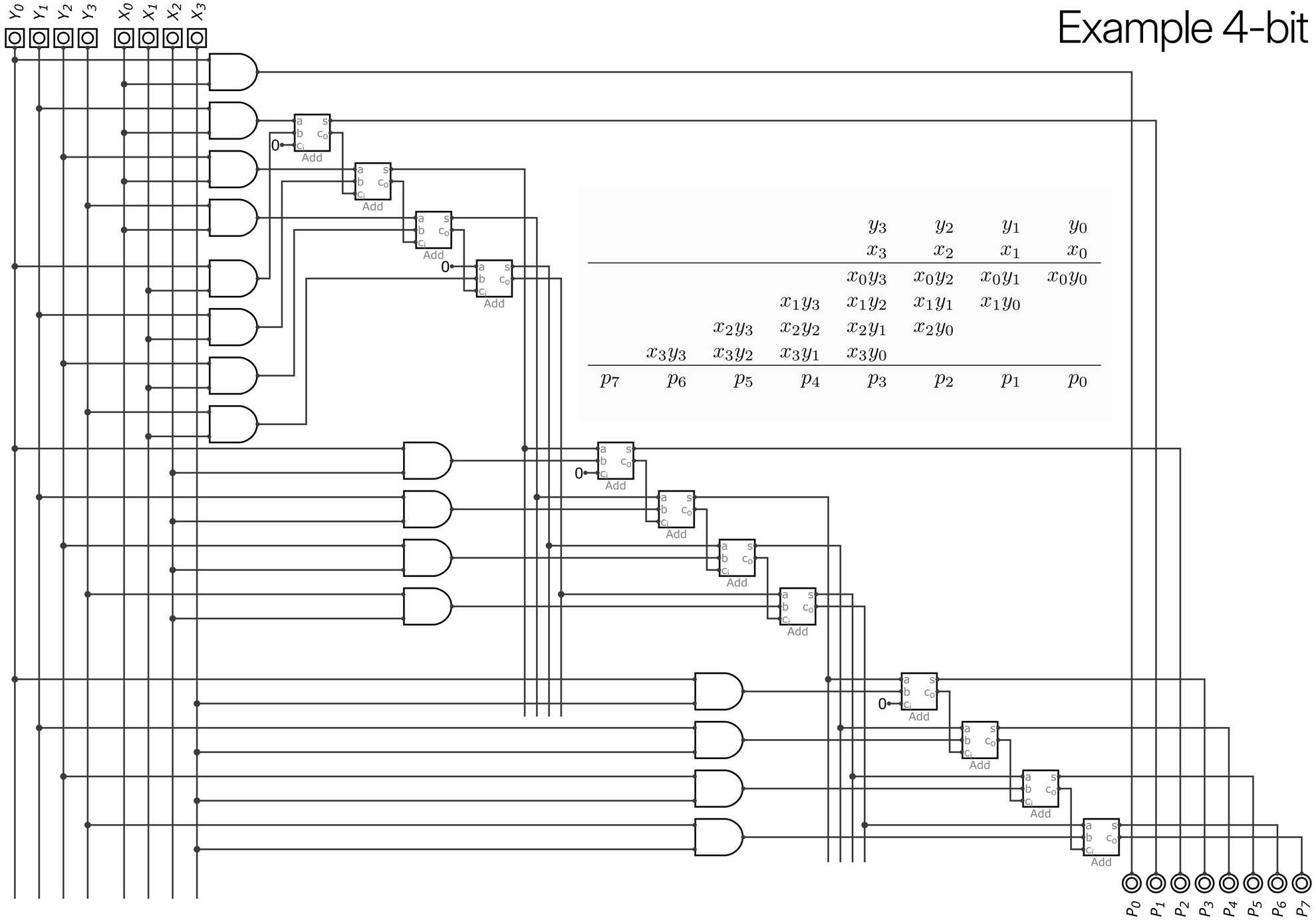
Multiplication

Decimal and Binary Multiplication

$$
\begin{array}{r}
743 \\
\times\ 201 \\
\hline
743 \\
000 \\
+\ 1486 \\
\hline
149343
\end{array}
$$

$$
\begin{array}{r}
1011100111 \\
\times\quad 11001001 \\
\hline
1011100111 \\
0000000000 \\
0000000000 \\
1011100111 \\
0000000000 \\
0000000000 \\
1011100111 \\
+\ 1011100111 \\
\hline
1001000111010111111
\end{array}
$$

Example 4-bit multiplier



				y_3	y_2	y_1	y_0
				x_3	x_2	x_1	x_0
				x_0y_3	x_0y_2	x_0y_1	x_0y_0
			x_1y_3	x_1y_2	x_1y_1	x_1y_0	
		x_2y_3	x_2y_2	x_2y_1	x_2y_0		
	x_3y_3	x_3y_2	x_3y_1	x_3y_0			
p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

Summary and Outlook

- Had a look at some essential combinational logic: multiplexers, shifters, decoders, adders, multipliers
- We may reduce the delays of our circuits by formulating them in a smart way; this often involves more area / power → tradeoffs
- It's time to bring our circuits to life: hardware description language SystemVerilog (no more GUI)
- After that:
  - Arithmetic Logic Unit
  - Field Programmable Gate Arrays
  - Sequential Logic

```
module not_or_dataflow( input  logic i_a,
                                input  logic i_b,
                                output logic o_c );
  assign o_c = ~i_a | i_b;
endmodule
```

```
module not_or_dataflow_tb;
  logic l_a, l_b, l_c;

  not_or_dataflow dut( l_a,
                                l_b,
                                l_c );

  initial begin
    $dumpfile("dump_not_or_dataflow.vcd");
    $dumpvars;

    l_a = 1'b0;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b1 );

    l_a = 1'b1;
    l_b = 1'b0;
    #10;
    assert( l_c === 1'b0 );

    // more tests

    $finish;
  end
endmodule
```

SystemVerilog implementation of a simple module with a corresponding testbench.